

■ SESIÓN 1

Qué vamos a construir (sin código)

1. Idea central de la sesión

Al final de esta hora debe quedar claro:

*"Una app no es un juego grande. Es un **sistema** que contiene varios juegos. Y antes de escribir una sola línea de código, tenemos que **decidir** qué partes tiene ese sistema y cómo se relacionan."*

Esta sesión no tiene código. Es **100% pensamiento y diseño**.

2. Conexión con conocimientos previos

Pregunta inicial: "Cuando hiciste juegos en Scratch, ¿empezabas directamente arrastrando bloques o primero pensabas qué iba a pasar?"

Scratch	Esta app
Un proyecto = un juego	Un proyecto = varios juegos
Escenarios y sprites mezclados	Partes separadas y organizadas
Todo en un mismo sitio	Sistema con módulos

Lo que SÍ reutilizamos de Scratch:

- Pensar en "estados" (jugando, ganado, perdido)
- Eventos (clic, tecla)
- Lógica (si... entonces...)

Lo NUEVO:

- Separar las partes del sistema
- Pensar en **responsabilidades** (qué hace cada cosa)
- Diseñar **antes** de construir

3. Explicación clara del concepto

¿Qué es un sistema?

Un **sistema** es un conjunto de partes que trabajan juntas para hacer algo.

Ejemplo cotidiano:

- Tu mochila no es "un objeto gigante"
- Tiene: cremalleras, bolsillos, correas
- Cada parte tiene una **responsabilidad**
- Juntas forman algo útil

Tres palabras que hay que distinguir:

1. **Juego:** Tic Tac Toe, Ahorcado (una actividad con reglas)
2. **App:** El programa completo que instalas
3. **Sistema:** Cómo están organizadas las partes por dentro

No son sinónimos.

4. Diseño antes de código

Actividad principal (30 minutos)

Materiales: Papel, lápiz/iPad, colores

"Dibuja cómo crees que es esta app por dentro. No código, solo cajas y flechas."

Preguntas guía mientras dibuja:

1. "Cuando abres la app, ¿qué es lo primero que ves?" → Menú principal
2. "Si eliges Tic Tac Toe, ¿desaparece el menú o sigue ahí?" → Pensar en navegación
3. "Cuando terminas el juego, ¿cómo vuelves al menú?" → Flujo entre pantallas
4. "Si la app tiene 3 juegos, ¿comparten algo o cada uno es totalmente distinto?" → Partes comunes vs. específicas

Resultado esperado (ejemplo):

[illegible]

5. Ejemplo aplicado al proyecto

Las partes básicas que tendrá nuestra app:

Parte	Responsabilidad	Qué NO hace
Menú principal	Mostrar opciones y permitir elegir	Contener los juegos
Cada juego	Sus propias reglas	Saber qué otros juegos existen
Navegación	Cambiar entre menú y juegos	Jugar por ti

"Si mañana quieres añadir un cuarto juego, ¿tendrías que rehacer todo o solo añadir una pieza nueva?"

Si la respuesta es "solo una pieza nueva", el diseño es bueno.

6. Uso guiado de agentes de IA

Esta semana: NO usar IA todavía

Razón pedagógica: Si usas IA antes de saber qué quieres, te dará código que funciona pero no entenderás.

Próximas semanas: qué le pediremos

Ejemplos buenos (cuando llegue el momento):

- "Explícame qué diferencia hay entre estos dos diseños"
- "Dame dos formas distintas de organizar esto"

Ejemplos malos:

- "Hazme la app completa"
- "Genera el código del menú"

"La IA es un ayudante, no el arquitecto. Tú diseñas, ella construye."

7. Preguntas de pensamiento crítico

No son para responder de memoria, sino para **pensar**:

1. "Si tu app tuviera 10 juegos en lugar de 3, ¿tu diseño seguiría funcionando?"
2. "¿Qué pasaría si quisieras cambiar el color del menú? ¿Tendrías que tocar los juegos?"
3. "¿Es mejor tener un archivo gigante con todo o muchos archivos pequeños? ¿Por qué?"
4. "Si alguien más quisiera añadir un juego a tu app, ¿podría hacerlo sin romper nada?"

No hace falta responderlas todas hoy. Son semillas para ir reflexionando.

8. Mini-reto para esta hora

Antes de terminar la sesión, debes tener:

1. Un dibujo (digital o en papel) del sistema con:
 - Menú
 - Al menos 2 juegos
 - Flechas de navegación

2. Una lista escrita de:

- Qué hace cada parte
- Qué **NO** hace cada parte

3. Responder por escrito:

- "¿Por qué esta app NO es un solo juego grande?"

Criterio de éxito: Si puedes explicarle tu diseño a alguien que no sabe nada de programación y lo entiende, lo has hecho bien.

9. Qué NO hacer

Errores comunes de principiantes:

- Pensar: "Ya lo haré bonito después" → El diseño no es decoración, es estructura
- "Empiezo a programar y voy viendo" → Así se hacen sistemas caóticos
- "Esto es muy simple, no necesita diseño" → Incluso cosas simples necesitan estructura

Errores comunes de la IA (cuando la uses):

- Te dará código sin preguntarte qué quieres
- Mezclará todo en un solo archivo
- No te explicará **por qué** algo está diseñado así

■ Resumen ejecutivo (para la profesora)

Objetivo pedagógico real:

Que interiorice que **pensar antes de hacer** no es perder tiempo, es ganar claridad.

Señales de éxito hoy:

- Ha dibujado un sistema con partes diferenciadas
- Puede explicar qué hace cada parte
- Entiende que menú ≠ juego ≠ sistema

Señales de fracaso hoy:

- Solo quiere empezar a programar
- No ve diferencia entre "app" y "juego"
- Dice "ya lo entiendo" sin poder explicarlo

Si pasa esto: No avanzar. Quedarse en diseño hasta que haya claridad conceptual.

■ Frase final para la sesión

*"Hoy no has escrito código, pero has hecho lo más importante: **decidir qué vas a construir**. La mayoría de la gente salta este paso y luego se pierde. Tú no."*

Siguiente semana: Profundizaremos en esas partes que has dibujado hoy.

■ ■ Distribución del tiempo

- Explicación conceptual: 15 min
 - Dibujo y diseño: 30 min
 - Preguntas y cierre: 15 min
-
- Contenido viable para 1 hora
 - Sin código
 - Pensamiento crítico activado
 - Base sólida para todo el proyecto